



FILE PATH TRAVERSAL

VALIDATION OF START OF PATH

ACTIVIDAD 11

Indice

File Path Traversal: <i>Validation of start of path</i>	2
Especificaciones del laboratorio.....	2
Objetivo del laboratorio	2
Marco Teórico	3
Mecanismo del filtro y su bypass mediante validación de prefijo incompleta	3
Clasificación según OWASP Top 10 y triada CIA.....	3
Riesgos en un entorno real.....	3
Procedimiento	4
Explicación del Payload	7
Mitigación recomendada	8
Referencias.....	8

File Path Traversal: *Validation of start of path*

Especificaciones del laboratorio

Laboratorio: File Path Traversal, Traversal Sequences stripped with superfluous URL-decode

Nivel: Practitioner

Tipo de explotación: File Path Traversal, evasión de una validación de prefijo de ruta (startsWith) añadiendo secuencias de traversal después del directorio esperado.

Objetivo del laboratorio

Explotar una vulnerabilidad de path traversal presente en la funcionalidad de visualización de imágenes de producto, con el fin de recuperar el contenido del archivo `/etc/passwd` del servidor. A diferencia de las variantes anteriores, en este laboratorio la aplicación recibe la ruta completa del archivo a través del parámetro `filename` y valida que dicha ruta comience con el directorio esperado, `/var/www/images/`. El objetivo específico es construir un valor que satisfaga esa validación de prefijo y, al mismo tiempo, contenga una secuencia de traversal que permita salir de ese directorio.

Marco Teórico

Mecanismo del filtro y su bypass mediante validación de prefijo incompleta

Todas las variantes anteriores de este laboratorio recibían únicamente el nombre del archivo (por ejemplo, 15.jpg) y la aplicación se encargaba de anteponer el directorio base antes de acceder al archivo. Esta variante cambia el diseño, ya que el historial de proxy muestra que la aplicación envía y espera la ruta completa del archivo, incluyendo el directorio, como valor del parámetro (filename=/var/www/images/15.jpg). Como defensa, el servidor valida que ese valor comience literalmente con la cadena /var/www/images/, y si no es así, responde como si el parámetro faltara por completo.

PortSwigger documenta este obstáculo como una validación de prefijo (startsWith) que resulta insuficiente cuando el resto de la ruta, después del prefijo válido, no se sanea (PortSwigger, s.f.). El hecho de que un string comience con el directorio autorizado no garantiza que el resultado final, una vez resuelto por el sistema de archivos, se mantenga dentro de ese directorio.

Al enviar el valor /var/www/images/../../etc/passwd, el prefijo /var/www/images/ satisface la validación, por lo que la solicitud pasa esa comprobación sin problema. Sin embargo, el sistema de archivos no interpreta la ruta como un string estático, sino que resuelve cada componente en orden: primero entra a var/www/images, y luego cada ../ lo hace retroceder un nivel, hasta salir por completo de ese directorio y llegar a la raíz del sistema de archivos, desde donde etc/passwd es alcanzable. La validación de prefijo se cumplió en el string original, pero no dice nada sobre a dónde apunta realmente la ruta después de resolverse.

Clasificación según OWASP Top 10 y triada CIA

Al igual que en los casos anteriores, esta vulnerabilidad corresponde a la categoría A01:2025 – Broken Access Control del OWASP Top 10 de 2025, la categoría de mayor incidencia registrada, presente en el 100% de las aplicaciones evaluadas (OWASP Foundation, 2025), y sigue mapeada a CWE-22 (MITRE Corporation, 2024). La diferencia frente a los casos previos es el tipo de control implementado: aquí no se trata de eliminar o detectar secuencias de traversal, sino de restringir el directorio de origen mediante una comprobación de prefijo sobre el string recibido, un enfoque que verifica el punto de partida de la ruta, pero no su destino final.

El impacto sobre la triada CIA sigue siendo el mismo que en los casos anteriores: comprometer la Confidencialidad al exponer archivos internos del servidor. Lo que distingue este caso es que expone la diferencia entre validar la forma de un dato y validar su significado real, una ruta puede cumplir perfectamente la condición de "empezar con el directorio correcto" y, aun así, terminar apuntando a un archivo completamente distinto una vez que el sistema operativo resuelve los componentes ../ que le siguen.

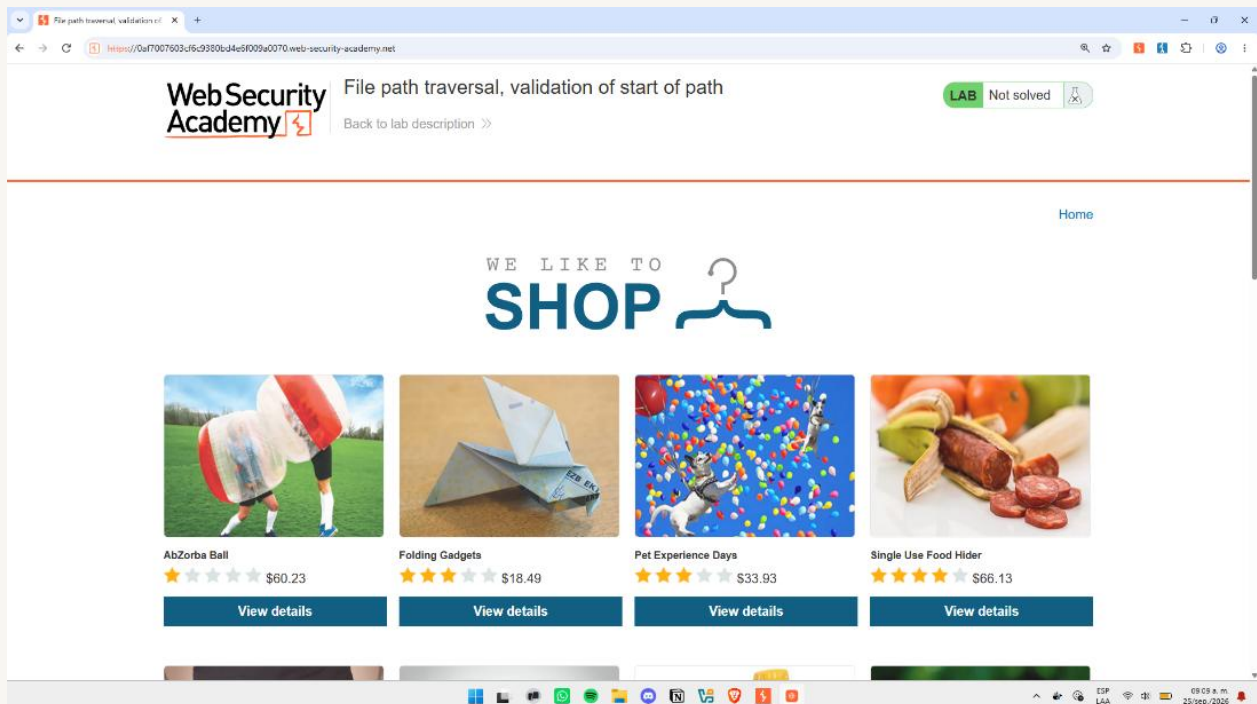
Riesgos en un entorno real

Validar únicamente el prefijo de una ruta es una práctica común en aplicaciones que reciben rutas de archivos completas desde el cliente, especialmente cuando se busca dar cierta flexibilidad al usuario final como por ejemplo, para acceder a subcarpetas dentro de un directorio compartido. El riesgo es que un desarrollador puede considerar suficiente esta comprobación por parecer sólida si se piensa que "si empieza con la carpeta permitida, debe estar dentro de la carpeta permitida", pero no se considera que el propio string puede incluir instrucciones

de navegación como los retrocesos en el directorio, que el sistema de archivos ejecutará después de que la validación ya haya aprobado el dato.

Procedimiento

1. Se accede a la instancia del laboratorio "File path traversal, validation of start of path" de PortSwigger Web Security Academy. El laboratorio inicia en estado "Not solved" y presenta una tienda en línea con productos cuyas imágenes se cargan dinámicamente.



2. Con Burp Suite interceptando el tráfico, se navega por la tienda y se revisa el historial de proxy. Se confirma que cada imagen de producto se solicita mediante GET /image?filename=<archivo>.jpg.

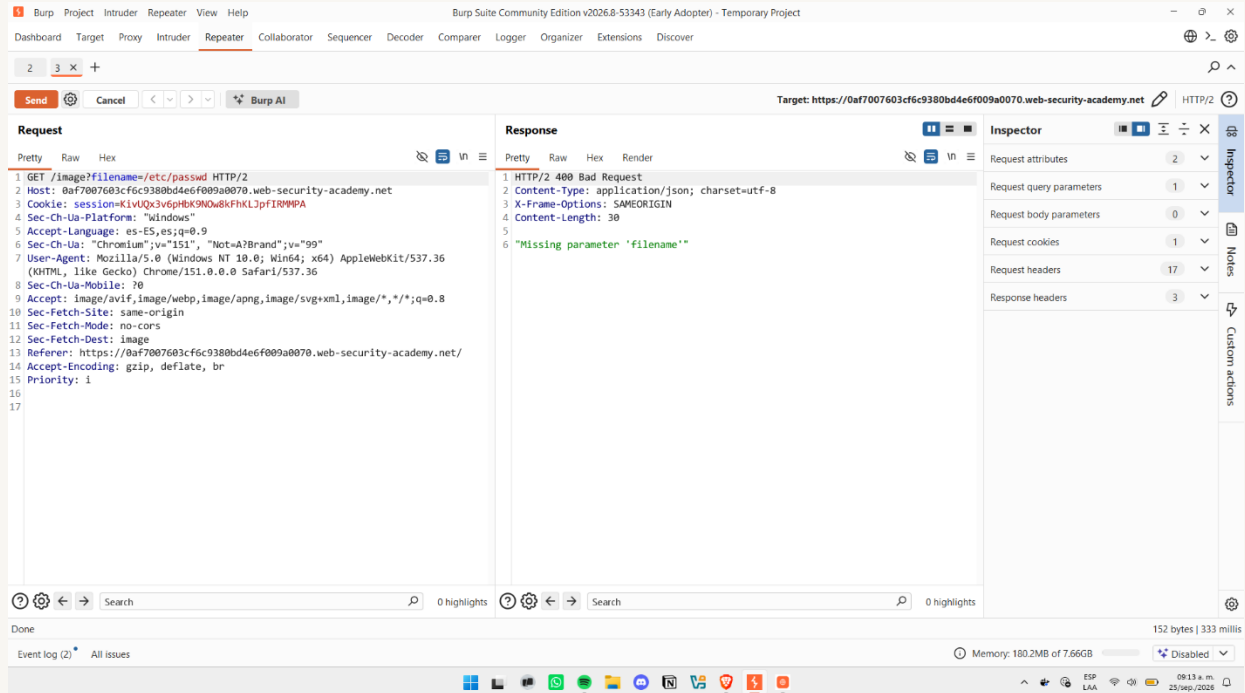
# ^	Host	Method	URL	Params	Edited	Status code
37	https://0af7007603cf6c9380...	GET	/			200
42	https://0af7007603cf6c9380...	GET	/image?filename=/var/www/images/33.jpg	✓		200
44	https://0af7007603cf6c9380...	GET	/image?filename=/var/www/images/8.jpg	✓		200
46	https://0af7007603cf6c9380...	GET	/image?filename=/var/www/images/15.jpg	✓		200
47	https://0af7007603cf6c9380...	GET	/image?filename=/var/www/images/20.jpg	✓		200
48	https://0af7007603cf6c9380...	GET	/image?filename=/var/www/images/38.jpg	✓		200
51	https://0af7007603cf6c9380...	GET	/image?filename=/var/www/images/25.jpg	✓		200
52	https://0af7007603cf6c9380...	GET	/image?filename=/var/www/images/36.jpg	✓		200
53	https://0af7007603cf6c9380...	GET	/image?filename=/var/www/images/64.jpg	✓		200
55	https://0af7007603cf6c9380...	GET	/academyLabHeader			101
58	https://0af7007603cf6c9380...	GET	/image?filename=/var/www/images/52.jpg	✓		200
59	https://0af7007603cf6c9380...	GET	/image?filename=/var/www/images/5.jpg	✓		200
60	https://0af7007603cf6c9380...	GET	/image?filename=/var/www/images/63.jpg	✓		200

3. Se selecciona una de las solicitudes de imagen en el historial y, mediante el menú contextual, se envía al módulo Repeater para poder modificar y reenviar la solicitud de forma controlada.

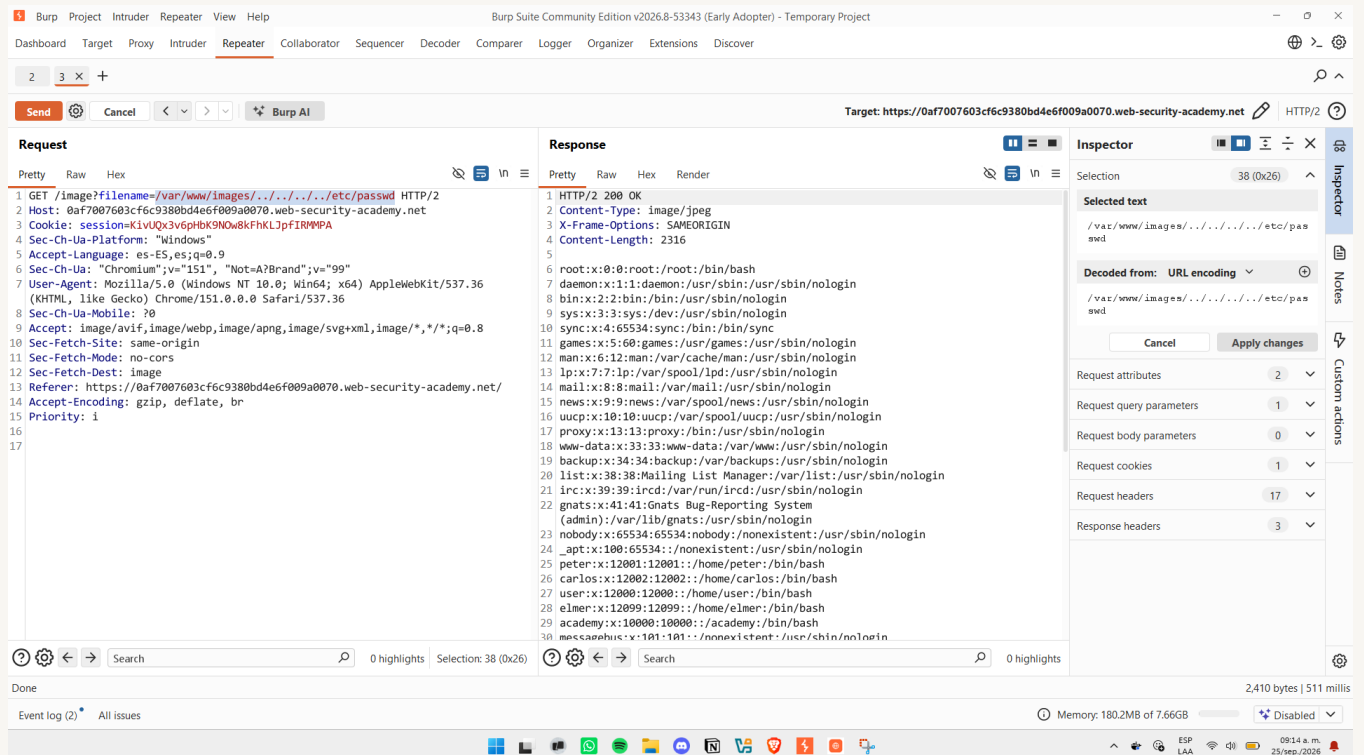


4. En Repeater se visualiza la solicitud original sin modificar:
GET /image?filename=/var/www/images/15.jpg, junto con las cabeceras enviadas por el navegador.

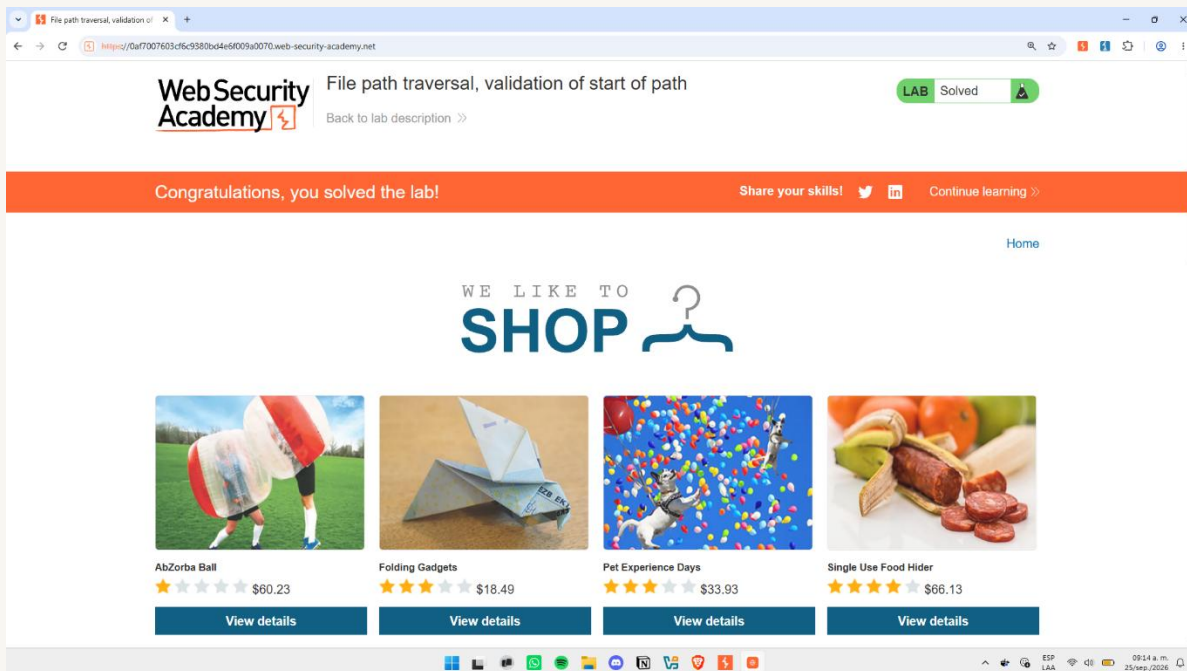
5. Se modifica filename con el valor /etc/passwd, sin el prefijo esperado. El servidor responde 400 Bad Request con el mensaje "Missing parameter 'filename'", lo que indica que la aplicación descarta por completo cualquier valor que no comience con /var/www/images/, tratándolo como si el parámetro nunca se hubiera enviado.



6. Se modifica filename con el valor `/var/www/images/../../etc/passwd`, que sí comienza con el prefijo exigido por la validación pero continúa con secuencias de traversal. Al enviar la solicitud, el servidor responde 200 OK con el contenido completo del archivo `/etc/passwd`.



7. Al regresar a la aplicación, el laboratorio muestra el estado "Solved" junto con el mensaje "Congratulations, you solved the lab!", confirmando que la explotación fue exitosa.



Explicación del Payload

El payload utilizado fue `/var/www/images/../../../../etc/passwd`, colocado como valor completo del parámetro filename. A diferencia de intentar ocultar o codificar el patrón `"../"`, esta vez el payload lo expone tal cual, la estrategia no trata de evadir la detección de la secuencia de traversal, sino de satisfacer la condición que que revisa que el valor recibido comience con el directorio que está autorizado.

Su efectividad depende de que la validación de la aplicación se limite a comprobar el inicio del string, algo equivalente a una función `startsWith("/var/www/images/")`, sin canonicalizar ni resolver el resto de la ruta. El payload cumple esa condición literalmente ya que los primeros caracteres de la cadena son exactamente `/var/www/images/`. Aprobada esa comprobación, el valor completo se entrega a la función del sistema de archivos que sí interpreta cada segmento de la ruta, incluyendo los `../` que siguen al prefijo, y que la llevan a salir del directorio de imágenes y llegar hasta `/etc/passwd`.

La validación de esta aplicación no tiene ningún defecto al comprobar el prefijo, en efecto, cualquier valor que no comience con `/var/www/images/` es rechazado, como se confirmó en el paso 5. La debilidad de este recae en que al verificar si el inicio es o no correcto, si no se sanitiza correctamente, el usuario aun puede moverse a través de los directorio.

Mitigación recomendada

- **No validar solo el prefijo del string recibido:** una comprobación de tipo `startsWith()` sobre una ruta sin resolver no es una validación de seguridad confiable, ya que no tiene en cuenta los componentes de navegación (`../`) que puedan aparecer después del prefijo aceptado.
- **Canonicalizar antes de realizar la validación:** resolver la ruta completa a su forma absoluta y canónica mediante funciones nativas del lenguaje (`getCanonicalPath()` en Java, `os.path.realpath()` / `Path.resolve()` en Python) y verificar el prefijo únicamente sobre ese resultado ya resuelto, nunca sobre el string tal como llegó del cliente.
- **Evitar que el cliente controle la ruta completa:** en lugar de recibir el directorio y el nombre de archivo en un mismo parámetro, separar ambos valores, fijar el directorio base en el servidor, sin que el cliente pueda influir en él, y aceptar del cliente únicamente el nombre del archivo, validado además contra una whitelist de caracteres o extensiones permitidas.
- **Verificar la relación de contención, no solo la coincidencia de texto:** tras canonicalizar la ruta, confirmar explícitamente que el resultado se encuentra dentro del directorio autorizado, por ejemplo, comparando que la ruta canónica del archivo comparta el mismo directorio padre autorizado), en lugar de asumir que un prefijo textual correcto implica una ubicación final correcta.
- **Principio de menor privilegio:** ejecutar el proceso del servidor con permisos mínimos, de forma que, aunque una ruta lograra evadir la validación de la aplicación, el sistema operativo continúe negando el acceso a directorios sensibles como `/etc` o `/root`.
- **Auditoría y pruebas específicas de bypass:** al probar validaciones basadas en el directorio de origen, incluir explícitamente payloads que combinen el prefijo esperado con secuencias de traversal, además de las variantes como rutas absolutas, secuencias anidadas, doble codificación URL.

Referencias

MITRE Corporation. (2024). CWE-22: Improper limitation of a pathname to a restricted directory ('Path Traversal'). Common Weakness Enumeration. <https://cwe.mitre.org/data/definitions/22.html>

OWASP Foundation. (2025). A01:2025 – Broken Access Control. OWASP Top 10:2025. https://owasp.org/Top10/2025/A01_2025-Broken_Access_Control/

PortSwigger. (s. f.). Lab: File path traversal, validation of start of path. Web Security Academy. <https://portswigger.net/web-security/file-path-traversal/lab-validate-start-of-path>

PortSwigger. (s. f.). What is path traversal, and how to prevent it? Web Security Academy. <https://portswigger.net/web-security/file-path-traversal>